

Reflective Journal

Title: Deploying a Simple AI Model on a Simulated Edge Device using Visual Studio Code

During this lab I learned how the “edge AI” workflow connects training, conversion, and validation in a lightweight runtime. I started by setting up my VS Code environment with Python and TensorFlow, then loaded the MNIST dataset and normalized the pixel values so the model would train consistently. After reshaping the data into the (28×28×1) format required for a CNN, I trained a small convolutional network and converted it to TensorFlow Lite, generating model.tflite.

The most important part for me was validation. Instead of assuming the conversion worked, I ran local inference using a TFLite interpreter to confirm the model behaves correctly outside of the training environment. My test run showed 96.00% accuracy on 25 MNIST samples, which gave me confidence that the exported .tflite model is usable for edge-style deployment.

The main challenge was the Edge Impulse simulation step. I attempted to use the Edge Impulse CLI as instructed, but on Windows the daemon failed during authentication and crashed with a uv async assertion error. Even though I couldn't complete the Edge Impulse simulation, I documented the error with screenshots/logs and still validated the model using a local simulated runtime approach. If I had more time, I would try running the Edge Impulse CLI in WSL/Linux or using the Studio web workflow to complete the simulation portion.

Snippets / Evidence Included:

- Snippet (TFLite conversion): with open('model.tflite', 'wb') as f: f.write(tflite_model)
- Snippet (Validation output): TFLite accuracy on 25 samples: 96.00%